Sabthami Balasubramanian
24130060

# The Trouble with Gradients:

## Why RNNs Forget and How LSTMs Remember

If you have ever tried to train a recurrent neural network on a task that requires remembering something from more than a few dozen steps ago, you have probably run into a frustrating wall: the model just does not seem to learn what it should. The loss might go down, training looks fine on the surface, but the network is quietly ignoring anything that happened more than a short while back in the sequence. That is not a bug in your code, it is a fundamental mathematical problem called the vanishing gradient, and it has been quietly breaking sequence models since the early 1990s.

This tutorial walks you through exactly why this happens and shows how Long Short-Term Memory networks (LSTMs) were designed specifically to fix it. Rather than just talking about it in the abstract, we demonstrate everything directly using a sine wave prediction task. We train both an LSTM and a vanilla RNN on the same set of data, and we compare not only their final accuracy but also what happens to their gradients during training, which is where the real story lies.

By the end, you should understand why LSTMs are the preferred architecture for the majority of sequence tasks, have a clear mental model of the vanishing gradient problem, and develop the useful habit of verifying gradient behavior during training, which will save you a significant amount of debugging time in subsequent projects.

### 1. How RNNs Work and Where They Break Down

A Recurrent Neural Network is built around a simple idea: at each time step, the network takes in the current input and combines it with a hidden state from the previous step to produce a new hidden state. That hidden state is the network's memory , in theory, it can carry forward any information the model has seen so far.

The update rule looks like this:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

Training in RNNs uses **Backpropagation Through Time (BPTT)**, where the model is unrolled across time and the error is traced backwards to update the weights.

The problem starts here. As we go backwards, the gradient is multiplied by the same weight matrix again and again. In most cases, this repeated multiplication causes the gradient to shrink.

After around 20–30 steps, the gradient becomes so small that it is almost zero. When this happens, the model can no longer learn from earlier parts of the sequence

and effectively “forgets” long-term information.The network effectively stops learning from events more than a handful of steps in the past.

Bengio, Simard, and Frasconi (1994) were among the first to formalise this mathematically. They showed it is not a quirk of bad luck or poor initialisation, it is a structural consequence of how vanilla RNNs are trained. The exploding variant, where gradients grow instead of shrinking, is actually easier to deal with (gradient clipping solves it cleanly), but vanishing gradients are the more dangerous problem precisely because nothing looks broken from the outside.

## 2. The Experiment: Predicting a Multi-Frequency Sine Wave

To make this concrete, we need a task that genuinely requires memory. A single-frequency sine wave would be too easy, any model that has seen even a few steps can infer the next value from local patterns alone. Instead, this tutorial uses a dataset made from two overlapping sine waves with different frequencies, along with a small amount of noise:

$$signal = \sin(t) + 0.5 \cdot \sin(0.2t) + noise$$

The second wave changes much more slowly than the first. Because of this, the model needs to look back over many steps (around 30 or more) to capture the pattern. This makes it a good test for long-term memory, which is where basic RNNs tend to struggle.

Each sample consists of 50 consecutive values, and the task is to predict the next value in the sequence.

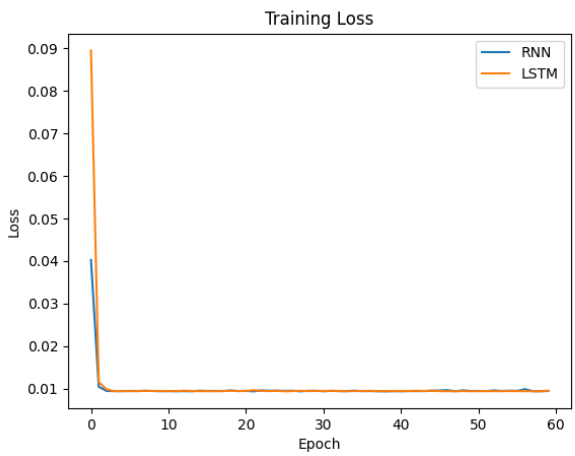


*Figure 1*

The dataset contains 2,000 samples in total. About 80% is used for training, while the remaining 20% is kept as a test set that the models never see during training.

## 3. The Two Models

Both models are deliberately kept as simple as possible so that any performance difference is clearly due to the architecture, not some difference in complexity or capacity.

### Vanilla RNN

The **SimpleRNN** model uses PyTorch's built-in nn.RNN layer with 32 hidden units. It processes the sequence one step at a time, updating its hidden state as it goes. After all 50 time steps are processed, only the final hidden state is used to make the prediction through a linear layer.

```
self.rnn = nn.RNN(input_size=1, hidden_size=32, batch_first=True)

self.fc  = nn.Linear(32, 1)

return self.fc(out[:, -1, :])
```

The important thing to note is that all past information has to be stored in this single hidden state as the sequence moves forward. There is no separate memory for long-term information.

### LSTM

The **SimpleLSTM** model is set up in almost the same way — same hidden size, same output layer, and it also uses only the final time step for prediction. The only difference is that nn.RNN is replaced with nn.LSTM:

```
self.lstm = nn.LSTM(input_size=1, hidden_size=32, batch_first=True)
```

This small change in architecture makes a big difference. Everything else — the data, training setup, and number of epochs — is kept the same so the comparison between the two models is fair and clear.

## 4. Why the LSTM Can Remember Longer

Hochreiter and Schmidhuber (1997) introduced LSTMs to solve the vanishing gradient problem. The key idea is that an LSTM keeps two types of information: the usual hidden state $h_t$, and an additional memory called the **cell state** $c_t$.

The hidden state works similarly to a standard RNN, but the cell state is different. Instead of updating through repeated multiplication, it is updated using addition, which helps preserve information over longer time steps:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

Here, the **forget gate** $f_t$, **input gate** $i_t$, and **output gate** $o_t$are learned functions that produce values between 0 and 1. These gates control what information is kept, what new information is added, and what is passed forward at each step.

What makes this powerful for gradient flow is that the cell state update is additive. When you unroll this backwards during BPTT, the gradient of c_t with respect to c_{t-1} is just f_t, a simple element-wise multiplication with the forget gate values, rather than a full matrix multiply raised to a power. As long as the forget gates stay close to 1 for the dimensions the model needs to track, gradients can flow backwards across many steps without exponentially decaying to zero. Hochreiter and Schmidhuber called this mechanism the "constant error carousel", and it is the structural reason LSTMs are able to learn dependencies that vanilla RNNs cannot.

### 5. Tracking Gradient Health During Training

One of the most useful things you can do when training sequence models is to monitor the gradient norm at each epoch. This is a single number that summarises how much information is flowing backwards through the network. The code in this tutorial records the total gradient norm per epoch by summing the gradient norms across all parameters after each backward pass:

```
def gradient_norm(model):
    return sum(p.grad.norm().item()
               for p in model.parameters()
               if p.grad is not None)
```

Both models are trained for 60 epochs using the Adam optimiser with a learning rate of 0.001 and Mean Squared Error as the loss function. The gradient norm is averaged over all batches within each epoch and stored alongside the loss.

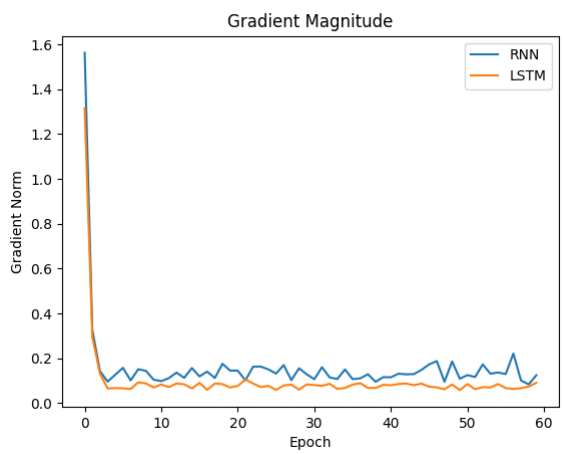


*Figure 2*

Figure 2 is the most important plot in this tutorial. Look at the RNN curve: within just a few epochs, the gradient norm drops sharply and then plateaus near zero. This tells us that the model has essentially stopped receiving useful training signal from the long-range patterns in the data. It is not updating randomly, it is just not learning to use context from more than a handful of steps ago.

The LSTM gradient norm behaves completely differently. It stays meaningfully above zero throughout all 60 epochs, reflecting the fact that the additive cell state is providing a stable gradient path back through the sequence. The model keeps improving because it keeps learning.

**6. Results: Predictions on the Test Set**

After training, both models are put into evaluation mode and run on the held-out test set. PyTorch's no_grad() context is used during inference to disable gradient tracking, which speeds up the forward pass and prevents accidental updates.

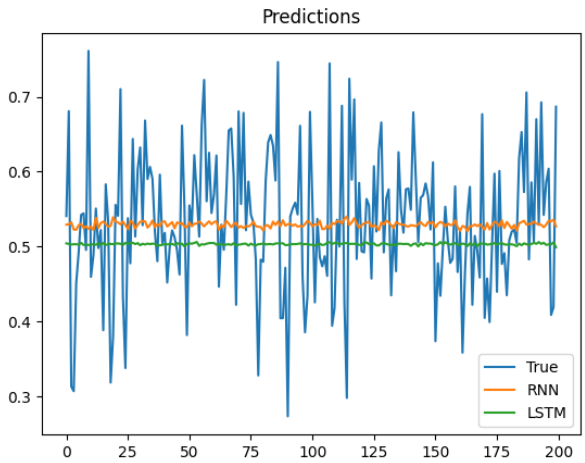


*Figure 3*

Figure 3 compares 200 steps of test predictions against the true signal. Over short stretches the two models can look similar, both are capturing the fast oscillation reasonably well. But look more carefully at the regions where the slow component of the signal is at a turning point. The RNN tends to get the phase wrong here, because it cannot reliably use context from 30 or more steps back. The LSTM tracks those transitions noticeably more accurately.

This matches what the gradient plots told us: the RNN's predictions are essentially being driven by local, short-term patterns. It has learned the shape of the fast sine wave and is extrapolating from that, while largely ignoring the slower underlying rhythm. The LSTM actually uses the longer context it was given, and the predictions show it.

## 7. A Note on Why This Matters in Practice

It is tempting to treat the vanishing gradient problem as a purely academic curiosity, but the consequences of getting it wrong in real applications can be serious. Sequence models of this kind are deployed in contexts where long-range dependencies genuinely matter, detecting irregular patterns in patient medical records, identifying fraud in sequences of financial transactions, or predicting failures in time-series sensor data from industrial equipment.

The insidious thing about a model silently suffering from vanishing gradients is that it can still look good on standard metrics. If your test set happens to be dominated by short-range patterns, a model that has completely given up on long-range context can still achieve acceptable loss values. It will only fail when it encounters the exact scenarios that require the longer memory, which might be the rarest and most important ones.

Two habits are worth building from the start. First, always track gradient norms alongside your loss during training, not just at the end. If you see norms collapsing to near-zero early on, that is your signal to investigate before investing more compute in hyperparameter tuning. Second, when constructing evaluation sets for sequence tasks, deliberately include examples that require long-range context and measure performance on those specifically. A model that looks fine on average can be silently broken on the cases that matter most.

## 8. Conclusion and Practical Takeaways

The vanishing gradient problem is not something you can engineer around inside a vanilla RNN. It is baked into the architecture, and it means that plain RNNs are simply not reliable tools for tasks where the important context stretches more than a few dozen steps into the past. The sine wave experiment in this tutorial makes this visible directly: the RNN's gradient collapses in the first few epochs, and its predictions reflect that by missing the slow component of the signal.

LSTMs solve this with a clean architectural idea: a cell state that updates additively, controlled by learned gates that determine what to keep, what to write, and what to expose. The result is that gradients can flow backwards through long sequences without being forced through repeated matrix multiplications that compress them to nothing.

The practical takeaway is straightforward. For any sequence task with dependencies longer than roughly 20 steps, start with an LSTM. The code change from a vanilla RNN is minimal, you replace nn.RNN with nn.LSTM and add initialisation of the cell state c0 alongside the hidden state h0. The performance difference, as this tutorial shows, can be substantial.

If you want to go further, GRU (Gated Recurrent Unit) is a close relative of the LSTM that uses fewer parameters and performs comparably on most tasks. For very long sequences or large-scale tasks, Transformer architectures take a completely different approach using self-attention rather than recurrence, and have become the standard for many sequence modelling domains. The tutorial's companion notebook includes all the code to reproduce these results and is a good starting point for experimenting with those ideas.

## References

Bengio, Y., Simard, P. and Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. IEEE Transactions on Neural Networks, 5(2), pp.157–166.

Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), pp.1735–1780.

Olah, C. (2015). Understanding LSTM Networks. Available at: https://colah.github.io/posts/2015-08-Understanding-LSTMs/ (Accessed: 26 March 2026).

Pascanu, R., Mikolov, T. and Bengio, Y. (2013). On the difficulty of training recurrent neural networks. Proceedings of the 30th International Conference on Machine Learning, pp.1310–1318.

PyTorch (2024). torch.nn Documentation. Available at: https://pytorch.org/docs/stable/nn.html (Accessed: 26 March 2026).